

# MACRAScript

COMPUTER SCIENCE III



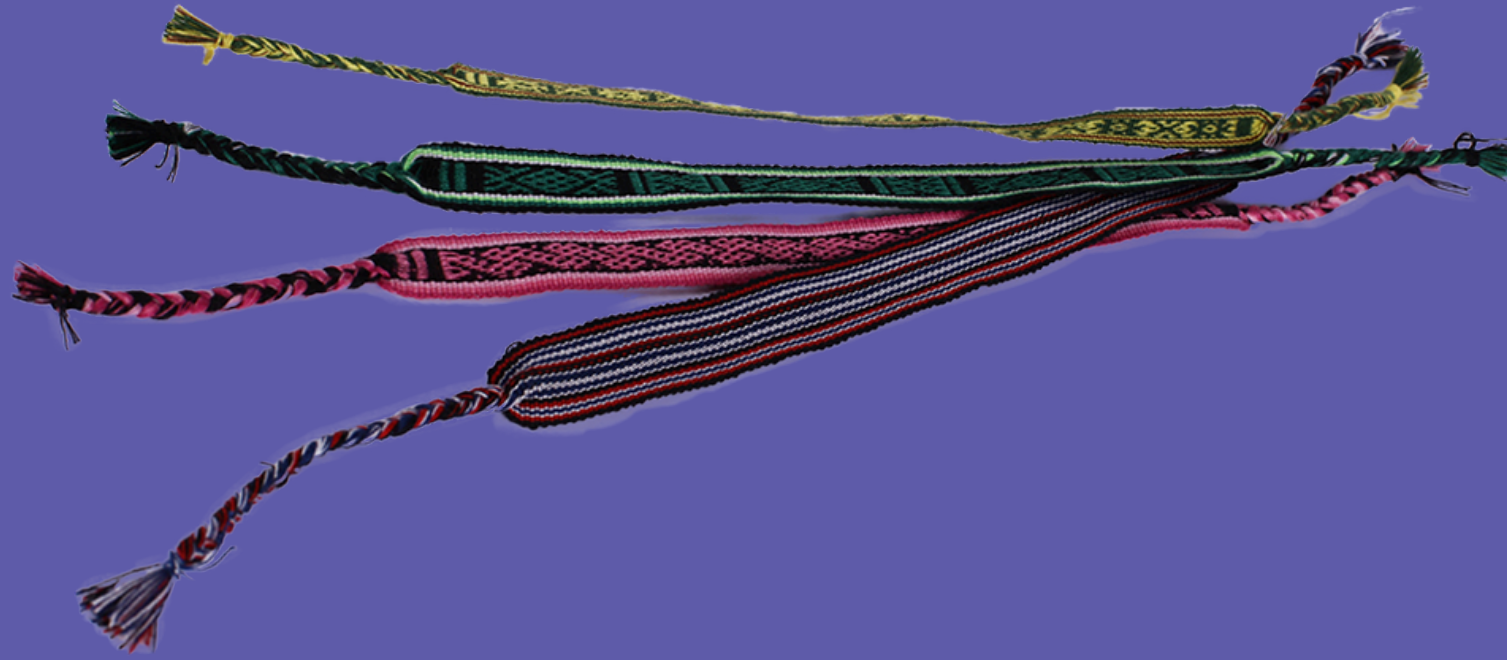
JUAN MANUEL SERRANO RODRÍGUEZ  
JAVIER ALEJANDRO SANCHEZ SALAMANCA

**DSL LANGUAGE FOR  
MACRAMÉ PATTERNS**

# INTRODUCTION

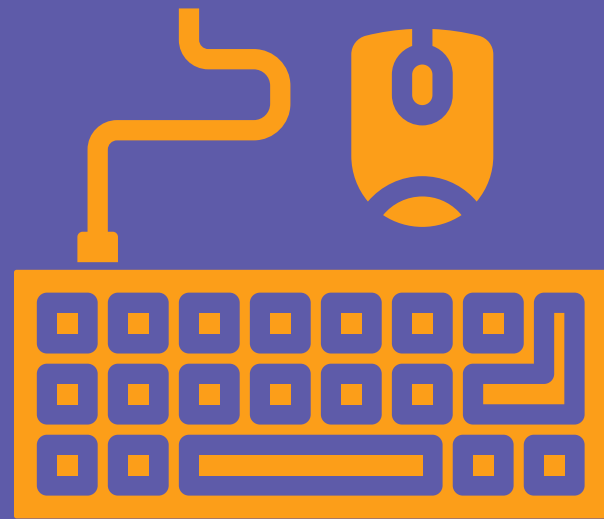
Macramé design is traditionally a manual, tedious, and error-prone process. Artisans use grid paper or generic software, which are not optimized for the specific needs of knot-based textiles. Our project was born from a simple question: "Can we do better?"





# PROJECT OBJECTIVES

- Create a Domain-Specific Language (DSL): MacraScript.
- Enable the design, visualization, and modification of macramé patterns through code.
- Facilitate pattern reuse and automatic validation.



# METHODS AND APPROACH

- Iterative design based on artisans' needs.
- Use of declarative syntax adapted to the domain.
- Data model to represent threads, colors, and knots.
- Visual outputs: preview.



# WHAT IS MACRAScript?

It's a complete computational system for designing, validating, and visualizing macramé patterns. It consists of:

- A Domain-Specific Language (DSL): A simple programming language we created specifically to describe macramé patterns.
- A Mini-IDE: A desktop application built with Python and Tkinter that provides a code editor, a compiler, and a graphical visualizer.



# THE GRAMMAR

The keyword after START – either ALPHA or NORMAL – tells the compiler how to interpret all subsequent ROW definitions.

- In ALPHA mode, a ROW contains a list of color indices. It's a direct, 1-to-1 mapping for creating images.
- In NORMAL mode, a ROW contains a list of knot directions (LEFT or RIGHT). This is a powerful abstraction, as the compiler handles the complex logic of which threads to knot.

# RESULTS

- Intuitive and expressive syntax.
- Allows representation of both simple and complex patterns.
- Reduces common errors and learning curve.
- Facilitates modifications and the creation of pattern libraries.

# THE COMPILATION FLOW

## PRACTICAL EXAMPLES

Let's trace a simple  
NORMAL script through  
each phase of the  
compiler.

```
MacraScript Code

START NORMAL
THREADS: 8
COLORS: ("red", "blue", "yellow", "green", "magenta", "cyan",
, "orange", "purple")

PATTERN {
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
}
END
```



# PHASE 1

# LEXICAL ANALYSIS

The lexer breaks the text into a list of tokens that the compiler can understand

```
MacraScript Code

START NORMAL
THREADS: 8
COLORS: ("red", "blue", "yellow", "green", "magenta", "cyan",
, "orange", "purple")

PATTERN {
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
}
END
```

```
--- 1. Lexical Analysis ---
Token(KEYWORDS, START)
Token(KEYWORDS, NORMAL)
Token(KEYWORDS, THREADS)
Token(COLON, :)
Token(INTEGER, 8)
Token(KEYWORDS, COLORS)
Token(COLON, :)
Token(LPAREN, ()
Token(COLOR_NAME, "red")
Token(COMMA, ,)
Token(COLOR_NAME, "blue")
Token(COMMA, ,)
Token(COLOR_NAME, "yellow")
Token(COMMA, ,)
Token(COLOR_NAME, "green")
```



# PHASE 2

# SYNTACTIC ANALYSIS

The parser validates the token structure against the grammar and organizes it into a simplified Abstract Syntax Tree, which is just a Python dictionary

```
--- 2. Syntactic Analysis ---  
Parsed Structure: {'type': 'NORMAL', 'threads': 8, 'colors': ['red',  
'magenta', 'cyan', 'orange', 'purple'], 'pattern_data': [['RIGHT',  
'LEFT', 'LEFT', 'LEFT'], ['RIGHT', 'RIGHT', 'RIGHT', 'RIGHT'], ['L',  
, 'RIGHT', 'RIGHT', 'RIGHT']]}  
-----
```

# PHASE 3

## SEMANTIC ANALYSIS

This is where the magic happens. The semantic analyzer understands the meaning. It sees the mode is NORMAL, validates the logic (e.g., thread count is even), and most importantly, it transforms the abstract row of directions into a concrete list of knot instructions. It knows the first row uses pairs (1,2) and (3,4), so it generates:

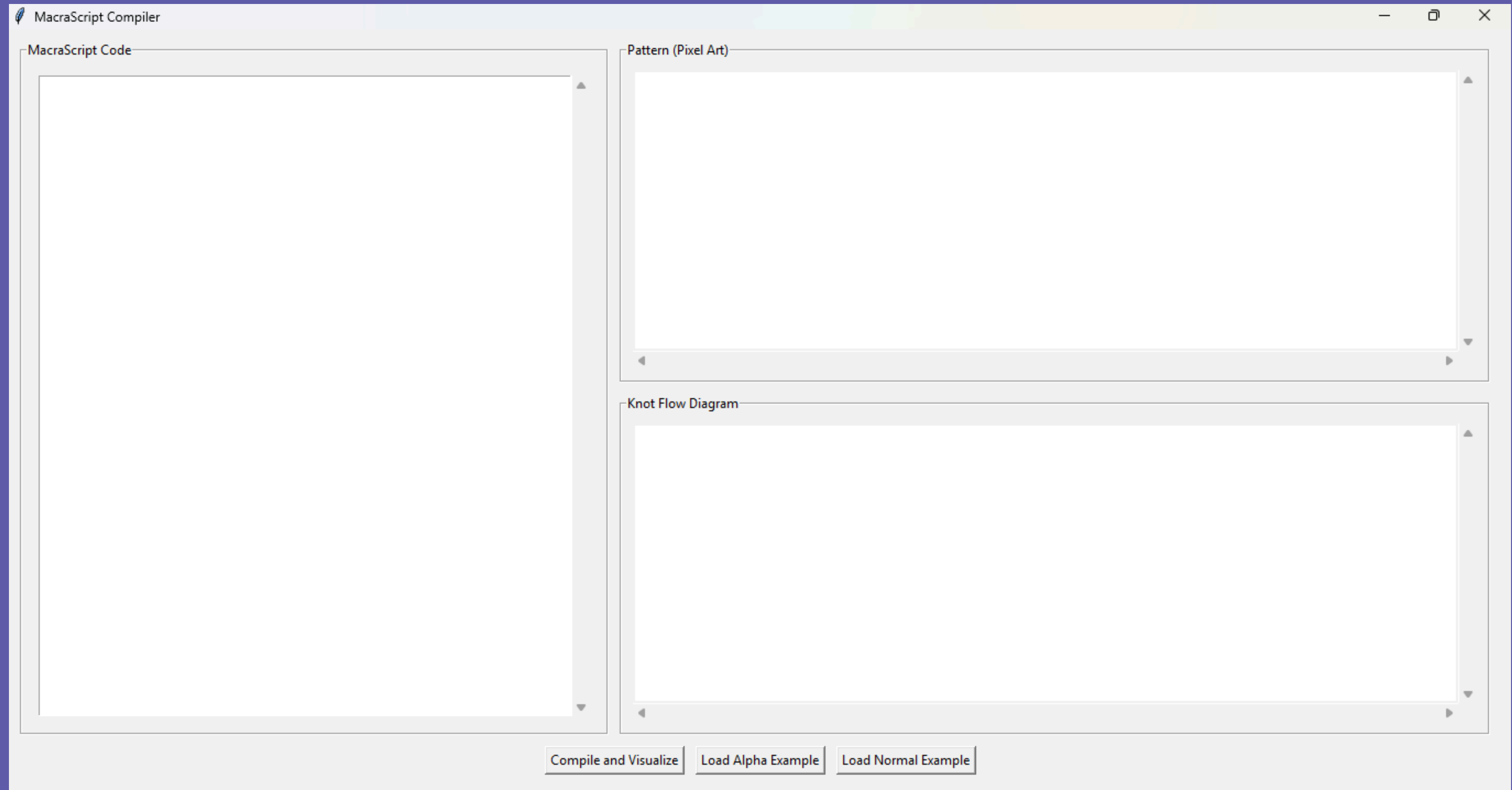
```
--- 3. Semantic Analysis ---
MacraPattern (NORMAL)
  Threads: 8
  Dimensions: 0x0
  Colors: 8 ['red', 'blue', 'yellow', 'green', 'magenta', 'cyan', 'orange', 'black']
  Knots: 18
  Preview:
    {'direction': 'RIGHT', 'threads': [1, 2], 'repeat': 1}
    {'direction': 'RIGHT', 'threads': [3, 4], 'repeat': 1}
    {'direction': 'RIGHT', 'threads': [5, 6], 'repeat': 1}
    {'direction': 'RIGHT', 'threads': [7, 8], 'repeat': 1}
    {'direction': 'LEFT', 'threads': [2, 3], 'repeat': 1}
    ...
  -----
```

# ANALYSIS ALPHA PHASES

The lexical and syntactic analyses are similar. The key difference is in the semantic analysis. For ALPHA mode, there's no complex transformation. The analyzer simply validates that the dimensions match and that the color indices are valid. The 'pattern data' is passed directly to the rendering engine.

```
--- 3. Semantic Analysis ---  
MacraPattern (ALPHA)  
  Threads: 4  
  Dimensions: 4x4  
  Colors: 2 ['red', 'blue']  
  Rows: 4  
  Preview:  
    [0, 1, 0, 1]  
    [1, 0, 1, 0]  
    [0, 1, 0, 1]  
    ...  
-----
```

# THE GUI AND RENDERING ENGINE



# GUI STRUCTURE AND CONTROLS

The GUI is built with Tkinter. The main buttons are:

**Compile and Visualize**

This is the main trigger. It runs the full compilation pipeline and, if successful, calls the drawing functions.

**Load Alpha Example**

**Load Normal Example**

These buttons simply load the content of the example text files into the editor.



# RENDERING LOGIC

- Pixel View (ALPHA): Draws a simple grid of colored rectangles.
- Diamond View (NORMAL): Draws a grid of diamonds. Crucially, the color of each diamond is determined by the active thread in the knot, ensuring consistency with the diagram below.
- Knot-Flow (ALPHA): Draws a snake-like pattern of nodes, with connecting lines colored to match their source node.
- Knot-Flow (NORMAL): This is the most complex algorithm. It iterates through the final knot list, drawing the crossing threads and placing a colored circle (representing the active thread's color) with a direction arrow at each intersection.

# CONCLUSION

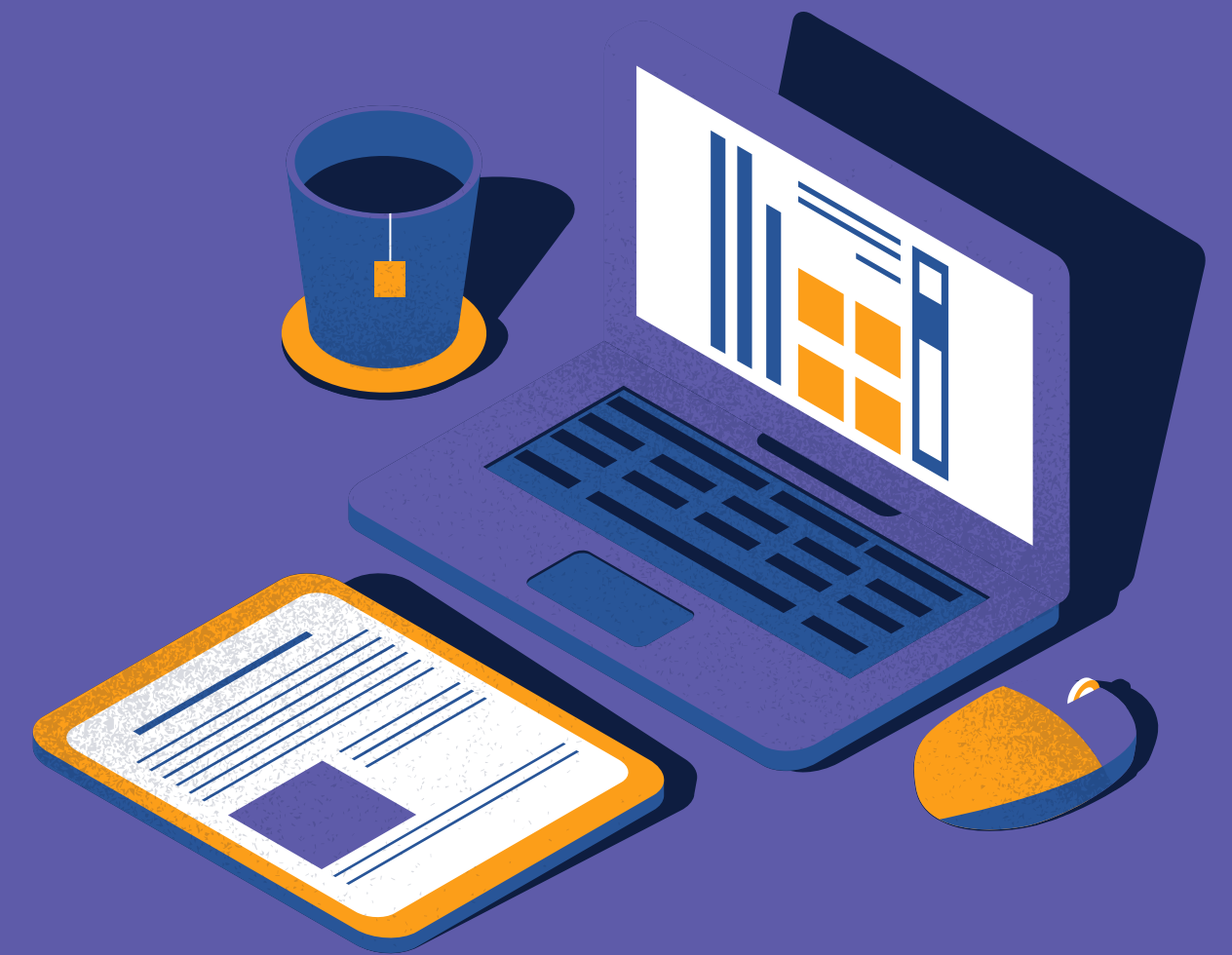
In summary, MacraScript is a complete system that successfully bridges the gap between a traditional craft and computational design.

We designed an intuitive, declarative DSL.

We implemented a full three-phase compiler.

The system generates dual, coherent, and accurate visualizations that serve as both a preview and a practical guide.

This project demonstrates the power of domain-specific languages to create powerful and accessible tools for specialized creative fields.







## REFERENCES

1. Fowler, M. Domain-Specific Languages.
2. Mernik et al. How to develop DSLs.
3. Rochford, K. Textile Patterns as Code.

# THANK YOU!